NEMOFLEET · PLATFORM FEEDBACK

# Feedback on Nebius Token Factory, Serverless AI and AI Cloud

Kept from day 1 of the build. Every item has the same four parts: **problem**, **measured** (a real number from our runs), **workaround**, and **suggested fix**. Items marked *to measure* are known issues we haven't put a number on yet.

## 1. Cosmos is available only as a dedicated endpoint

- **Problem:** `nvidia/Cosmos3-Super-Reasoner` is offered on Token Factory as a dedicated endpoint only, sized at 4× H200. Physical AI perception is bursty (in our design one call per new hazard), so paying for a whole always-on endpoint is a poor fit. Earlier research couldn't find Cosmos on Token Factory at all, so it's also hard to discover.
- **Measured:** $0.37 per minute (about $22.20 per hour) while running, us-central1, from the console. Preflight on Oct 7 got HTTP 429 "model is starting, please retry" from the endpoint, so a stopped endpoint can't serve a demo on demand. Every request we sent from Oct 7 02:23 UTC to 21:38 UTC (over 19 hours, every check) got the same 429 "starting", with no ETA or state in the response, and the dedicated endpoint does not appear in `GET /v1/models`, so a client can't tell "warming up" from "stuck" or "no capacity". *To measure:* cold-start time from stopped to Ready once it succeeds, and calls per demo run.
- **Workaround:** run the endpoint only during test and recording sessions, with min replicas at 0 or the endpoint stopped otherwise, and batch perception requests.
- **Suggested fix:** return endpoint state and an ETA (or a capacity error) instead of a bare 429 while starting, and list dedicated endpoints in `/v1/models` with their status. Offer Cosmos Reasoner per token (serverless), or a smaller single-GPU configuration with scale-to-zero, and list it in the model catalog next to Nemotron.

## 2. Nemotron models are not fine-tunable on Token Factory

- **Problem:** the fine-tunable list doesn't include Nemotron, so the hackathon's headline model family can't be post-trained on Token Factory.
- **Measured:** *to measure:* engineering time to build our own LoRA container, and the job wall time on Serverless Jobs.
- **Workaround:** LoRA in our own container on Serverless Jobs (H200), served by vLLM on a Serverless Endpoint.
- **Suggested fix:** add Nemotron Nano to Token Factory LoRA fine-tuning, with serving on the same platform.

## 3. Batch inference only promises results within 24 hours

- **Problem:** `/v1/batches` only commits to a 24 h completion window, which is too slow to iterate on during a 3-week build or to show live.
- **Measured:** *to measure:* turnaround of our week-1 test batch and of each Night Shift batch.
- **Workaround:** real overnight batch, plus concurrent per-token calls for the demo, labeled as such.
- **Suggested fix:** a faster batch tier (for example, under 1 h) or a published turnaround estimate per model.

## 4. Custom weights on dedicated endpoints are beta and available only on request

- **Problem:** the documented way to serve our own fine-tuned weights on Token Factory needs access approval, with no stated turnaround.
- **Measured:** *to measure:* days from request to access (requested on day 1).
- **Workaround:** a vLLM Serverless Endpoint.
- **Suggested fix:** self-serve custom weights (or LoRA adapters) for open NVIDIA models.

## 5. Serverless Jobs: cold start and GPU quota

- **Problem:** *to measure:* cold-start time for our container on CPU and GPU, time to get H200 quota, and how logs behave across a 20–50 job fan-out.
- **Measured:** *to measure.*
- **Workaround:** *to be determined.*
- **Suggested fix:** *to be determined*; possibly a first-class fan-out or array-job primitive.

## 6. Responses API silently ignores some familiar parameters

- **Problem:** on `POST /v1/responses`, `response_format: {type: "json_object"}` and `max_completion_tokens` are accepted with HTTP 200 but have no effect. A prose prompt with `response_format` still returned prose; a 60-token `max_completion_tokens` cap returned 4,127 tokens.
- **Measured (2026-10-08, us-central1, Nemotron 3 Ultra):** `text.format` with `type: "json_schema"` and `strict: true` is enforced (JSON returned even against a prose instruction). `max_output_tokens` is enforced.
- **Workaround:** use `text.format` and `max_output_tokens`.
- **Suggested fix:** reject unknown or Chat-Completions-only parameters with HTTP 400, or document them as ignored.

## 7. `max_output_tokens` includes reasoning tokens, and a low cap returns nothing

- **Problem:** with `max_output_tokens: 60`, Ultra spent all 60 tokens reasoning and returned `status: "incomplete"` with no message item at all.
- **Measured:** our decision prompts use 190–380 reasoning tokens; a 4,000-token cap left plenty of headroom and every decision completed.
- **Workaround:** set the cap well above typical reasoning use, and treat `incomplete` as invalid.
- **Suggested fix:** document that the cap covers reasoning, or offer a separate reasoning budget.

## 8. `reasoning.effort` had no measurable effect on Nemotron 3 Ultra

- **Measured:** on one hard prompt, reasoning tokens were 4,594 with no setting, 5,934 with `low` and 6,747 with `high`; latency did not drop with `low`. Single samples, so indicative only.
- **Suggested fix:** document whether `reasoning.effort` is supported per model.

## 9. Cosmos3 dedicated endpoint stuck in "starting"

- **Problem:** the `Cosmos3-Super-Reasoner` dedicated endpoint answered HTTP 429 "is starting, please retry" for more than a day.
- **Workaround:** the perception model is a config switch (`tools/use_perception.cjs`); Gemma 4 on a Serverless Endpoint served perception meanwhile.
- **Suggested fix:** show a start-up ETA or a failure reason instead of an indefinite "starting".

## 10. No vision-capable NVIDIA model on serverless Token Factory; Gemma 4 not served

- **Problem:** Nemotron Ultra, Super, Nano and Lightning all return "This model does not support image input". Gemma 4 is not in any region's catalog (only `gemma-3-27b-it`).
- **Workaround:** Gemma 4 26B-A4B on vLLM v0.30.0 on a Nebius Serverless AI Endpoint (one H200). It worked on the first try: about 15 minutes from create to answering, then 0.7–1.2 s per frame.
- **Suggested fix:** serve a vision model (Cosmos Reason or Gemma 4) serverless on Token Factory.

## 11. Serverless Jobs: what worked

- **Measured:** `nebius ai job create --dry-run` validated our job spec before spending anything, and logs streamed with `nebius ai job logs` while the job ran. Our first Cosmos-Predict2.5 job failed in setup because the upstream repo pins Python 3.13 while its CUDA 12.8 flash-attn wheels are Python 3.10 only. That one is not a Nebius issue, and the failure was easy to read in the job logs.